STUDENT CASE STUDY 4: SECURITY & ACCESS CONTROL INTEGRATION

Course: CMU-CS 445 – *System Integration Practices*
Semester: 2
Academic: 2024–2025
Instructor: MSc. Nguyen Dang Quang Huy

1. Objective

The purpose of this case study is to secure the integrated API system developed in Case Study 3 by implementing **authentication**, **authorization**, and **auditing** mechanisms.

Students must ensure that all API endpoints are protected against unauthorized access, that each role has the correct access permissions, and that all security events are logged and monitored.

This assignment emphasizes practical understanding of **secure system integration**, **role-based access control**, and **data protection standards** aligned with modern enterprise practices.

2. Scenario Background

After building functional APIs in Case Study 3, Company X requires the development of a **comprehensive security layer** to protect both HR (HUMAN_2025) and Payroll (PAYROLL) systems.

The CEO mandates that the integrated system must:

1. Authenticate and authorize users properly before data access.
2. Ensure data confidentiality and integrity across departments.
3. Provide complete auditing and activity tracking to meet compliance standards (ISO 27001, GDPR).

Students will implement these requirements by extending the existing API backend with secure authentication, role-based access control (RBAC), and audit logging features.

3. Tasks and Requirements

3.1 Authentication System

Students must implement **token-based authentication** using either **JWT (JSON Web Token)** or **OAuth2**.

Functional requirements:

- API endpoints:
  - POST /login – Validate user credentials and issue an access token.
  - POST /logout – Invalidate or blacklist the token.
- Passwords must be hashed using **bcrypt** or a similar secure algorithm.
- Tokens must have expiration (30–60 minutes).
- Refresh tokens can be implemented for session renewal.
- All API requests must use **HTTPS/TLS 1.3** for encrypted transmission.

3.2 Authorization System (Role-Based Access Control)

Implement **RBAC (Role-Based Access Control)** to enforce permission boundaries between user types.

User Roles and Access Rules:

- **Admin:** Full access to all API endpoints.
- **HR Manager:**
  - Can manage employees in HUMAN_2025.
  - Cannot modify payroll tables.
- **Payroll Manager:**
  - Can manage payroll, attendance, and bonus data in PAYROLL.
  - Cannot modify employee records.
- **Employee:**
  - Can only view personal profile and salary details.

Access control implementation:

- Middleware or decorators must validate both token and user role before allowing access.
- Unauthorized access attempts must return appropriate responses:

- 401 Unauthorized – Missing or invalid token.
- 403 Forbidden – Token valid but insufficient permissions.

Examples:

- Employees can only access GET /employees/{id} where {id} matches their account.
- HR Managers can access POST /add-employee and PUT /update-employee but not payroll APIs.
- Payroll Managers can access PUT /update-payroll but not modify employee data.

3.3 Auditing and Logging Requirements

To ensure **traceability** and **accountability**, every authentication and authorization event must be logged.

Audit log content:

- Timestamp of the event.
- Username or user ID.
- Action performed (login, logout, add, update, delete, view).
- Target resource (e.g., /employees, /payroll).
- Result (success, failure, or access denied).

Implementation requirements:

- Logs should be stored in a database table (e.g., SECURITY_LOGS) or centralized logging system (e.g., **ELK Stack**, **Azure Application Insights**, or **Grafana Loki**).
- Sensitive fields (e.g., password) must never be logged.
- Implement log rotation or scheduled cleanup (e.g., 90-day retention policy).
- System should be able to generate reports on login attempts and failed access for auditing purposes.

3.4 Security Testing and Validation

Students must perform both **functional testing** and **penetration testing** to validate system security.

Functional tests:

- Verify token issuance, expiration, and revocation.
- Check proper role enforcement (HR cannot modify payroll, etc.).
- Test unauthorized access and confirm correct HTTP responses.

Security tests:

- Attempt SQL injection or broken authentication simulation.
- Use **OWASP ZAP** or **Burp Suite** to analyze API vulnerabilities.
- Validate HTTPS enforcement and token protection against replay attacks.

Test reporting:

- Include test cases, expected outcomes, and screenshots from tools.
- Document all detected issues and resolutions.

3.5 Advanced Security Extensions (Optional for Bonus Points)

Students who wish to go beyond core requirements can implement **extended features** to improve real-world readiness:

3.5.1. Multi-Factor Authentication (MFA):

- *Implement time-based one-time password (TOTP) using Google Authenticator or Authy API.*
- *Require MFA for Admin and Manager accounts during login.*

3.5.2. Single Sign-On (SSO) Integration:

- *Simulate login via enterprise identity provider (e.g., Azure AD, Okta) using OAuth2/OpenID Connect flow.*

3.5.3. IP Whitelisting and Access Restriction:

- *Restrict certain API endpoints (e.g., payroll modification) to corporate network IP ranges only.*

3.5.4. Security Monitoring Dashboard:

- *Build a small dashboard showing login statistics, access attempts, and failed logins using mock data or log queries.*

*These optional tasks demonstrate mastery of **modern enterprise-grade integration security**.*

4. Expected Deliverables

4.1 Document Deliverables

Document Name	Action (New / Update)	Detailed Description / Student Instructions
13. Security & Authorization Design Document	**New**	Design and implement user authentication using JWT or OAuth2. Describe how users log in, how tokens are generated and validated, and what each role can do (Admin, HR Manager, Payroll Manager, Employee). Include a diagram showing the security flow.
14. Updated API Design & Implementation Document	**Update from Case 3**	Add security middleware to protect each API. Clearly specify which endpoints require authentication and which are public. Example: "GET /employees → requires HR Manager or Admin role."
15. Updated Test Plan Document	**Update from Case 3**	Include testing plan for login/logout, token validation, and permission checks. Explain how you will test expired tokens or unauthorized access attempts.
16. Updated Test Case Document	**Update from Case 3**	Add security test cases, such as trying to access /payroll without a valid token or using a token from a different role. Record the expected "403 Forbidden" response.
17. Updated SRS Document (Version 3)	**Update from Case 3**	Include detailed security requirements (authentication, authorization, data protection). Add non-functional requirements related to confidentiality and audit logging.

4.2 Software Functional Deliverables

Software Functions to Develop	Description / Implementation Guidance
1. Authentication (JWT or OAuth2)	Build secure login system: • POST /login: validate credentials and issue JWT. • POST /logout: invalidate token.
2. Authorization Middleware	Enforce role-based access for APIs: • Admin: full access. • HR Manager: manage employee data only. • Payroll Manager: manage salary data only. • Employee: view personal data only. Apply middleware checks for all protected endpoints.
3. Secure API Operations	Update API routes to require valid token. Example: • GET /employees: allowed for Admin, HR Manager. • GET /payroll: allowed for Admin, Payroll Manager.
4. Security Testing	Test expired/invalid tokens, role violations, and access control. Log every failed attempt for auditing.

5. Evaluation Criteria

Criteria	Description	Weight
Authentication Implementation	JWT/OAuth2 functionality, token handling	20%
Role-Based Authorization	Correct enforcement of user permissions	25%
Auditing & Logging	Completeness, traceability, and secure storage	20%
Error Handling & Testing	Proper handling of invalid tokens, denied access, and test quality	20%
Documentation & Code Quality	Clarity, organization, and completeness	15%